Pipeline Walkthrough: From API to Findings

Companion to "Predicting College Football Upsets: A Statistical Research Study" · Tristan Morton · November 2026

This document traces the study end to end: how the data is acquired, how it is cleaned and audited, how it moves through the statistical models and the simulation, how the output is read, and how it becomes figures and a report. For each stage it names the tools used and the reason for choosing them. Every file and function named here exists in the repository, and every headline number comes from `results/results.json`.

The whole pipeline at a glance

```
CollegeFootballData REST API
        │  cfb/api.py      authenticate, retry, cache raw JSON  ──►  data/raw/*.json
        ▼
Extract + Transform
        │  cfb/etl.py      flatten, validate, join, derive      ──►  data/clean/games.parquet
        │                  integrity audit (every build)        ──►  results/audit.json
        ▼
Model + Evaluate
        │  cfb/models.py   H1–H4 tests, Brier/calibration, bootstrap
        ▼
Simulate
        │  cfb/simulate.py 10,000-trial Monte Carlo under two null models
        ▼
Verify
        │  run_study.py    bug_impact(), leakage_impact(): controlled reruns
        ▼
Report
           run_study.py    figures (matplotlib)                 ──►  results/figures/*.png
           report/         report.md → build_pdf.py → report.pdf
```

One command runs the whole study:

```
.venv/Scripts/python run_study.py --seasons 2019,2021-2024 --seed 42 --trials 10000
```

`run_study.py` calls each stage in order and writes every reported number to a single `results/results.json`. Run it twice with the same seed and the file is identical byte for byte.

1. Tools and why each was chosen

Tool	Version	Used for	Why this tool
Python	3.12	Everything	The standard language for statistical computing; one language from API calls to the PDF
requests	2.34	REST API calls	Simple, dependable HTTP with timeouts; the API needs nothing more
python-dotenv	1.2	Loading the API key	Keeps the secret in a git-ignored `.env` file, never in code
pandas	3.1	Tabular cleaning, joins, grouping	Columns are named and joins are explicit, so each cleaning rule is one readable line
pyarrow / Parquet	26.0	Clean dataset storage	Columnar, typed and compact; preserves data types exactly, unlike CSV
NumPy	2.5	Vectorized simulation	The 10,000 × 1,587 Monte Carlo draws run as array operations in seconds
statsmodels	0.15	Logistic and OLS regression	Gives inference (standard errors, p-values, likelihood) that hypothesis testing needs; scikit-learn does not
scikit-learn	1.9	Brier score, log loss, AUC, calibration curve	Standard, tested implementations of forecast-evaluation metrics
SciPy	1.18	Chi-square and normal distributions	p-values for the likelihood-ratio and Hosmer–Lemeshow tests; normal CDF for the margin model
matplotlib	3.11	Figures	Full control over static publication figures; renders headless (Agg backend)
pytest	9.2	Tests	20 tests pin the cleaning rules, the statistics and the leak guard
markdown + Microsoft Edge (headless)	3.11	Report PDF	Markdown is diffable in git; Edge is already on Windows, so no LaTeX install is needed
pypdf	6.21	Page count	Reads the finished PDF to report its length
Git + GitHub	—	Version control	Every change is a reviewed commit on a branch, merged through a pull request

Why statsmodels and scikit-learn both? They answer different questions. statsmodels fits the models and reports inference: is a coefficient different from zero, and do five features jointly add anything? scikit-learn scores the forecasts: how good are the probabilities on games the model never saw? A research study needs both.

Why not a database? The clean dataset is 3,900 rows. A Parquet file plus a raw-JSON cache is simpler, needs no server, and can be versioned and checksummed. A database would add operations work with no analytic benefit at this scale.

2. Extract: the REST client (`cfb/api.py`)

Source. CollegeFootballData.com is a free REST API serving college football data as JSON. Four endpoints are used:

Endpoint	Returns	Why it's needed
`/games`	One record per game: teams, scores, venue, conference, season type, team classification	The outcomes being predicted
`/lines`	Per game, a list of sportsbook lines: spread, over/under, moneylines	The market's forecast
`/teams/fbs`	That season's official FBS membership list	An independent check on which games qualify
`/ratings/sp`	SP+ team-strength ratings for each season	A non-market team-strength feature

How `get(endpoint, **params)` works: 1. **Cache first.** Each request maps to a file such as `data/raw/games_classification=fbs_seasonType=regular_year=2023.json`. If the file exists, it is read and the network is never touched. The first full run makes about 30 calls; every later run is offline and instant. 2. **Authenticate.** The key is loaded from `.env` and sent as `Authorization: Bearer <key>`. The key never appears in code, logs or git. If it is missing, the client stops with a clear message. 3. **Retry with backoff.** Rate limits (HTTP 429), server errors (5xx) and network failures are retried up to 5 times, with an exponential wait between tries (honoring the server's `Retry-After` header). 4. **Save on success only.** The raw JSON is written to the cache only after a successful response, so a failed call can never leave a half-written file.

Why cache the raw responses? Reproducibility. The API's data can change over time, for example when a sportsbook line is corrected. Caching freezes the exact inputs the study used. The clean dataset's SHA-256 checksum is recorded in `results.json`, so anyone can confirm they are analyzing the same data.

Choosing the seasons. `run_study.py --per-season-counts --seasons 2013-2024` prints the usable game count for each season. The 2019 and 2021–2024 seasons give 3,900 games, close to the target of about 4,000, and they are the most recent seasons. 2020 is excluded: COVID cut it to 534 games, and many were played without fans, which distorts home-field advantage.

3. Transform: cleaning and the integrity audit (`cfb/etl.py`)

`clean()` is a **pure function**: raw lists in, a clean table and an audit out, with no network or disk access. That makes every rule testable on small hand-built fixtures in `tests/test_etl.py`.

Step by step:

Step	What happens	Rows affected
1. Flatten games	`game_row()` maps the API's camelCase fields to snake_case columns, tolerating missing or renamed fields	4,501 raw game rows
2. Deduplicate	Drop repeated game IDs, and repeated (game, sportsbook) lines	0 duplicates found
3. Validate lines	`line_rows()` flattens 17,601 sportsbook lines and coerces spreads to numbers; null, non-numeric and impossible spreads (over 70 points) are counted and set aside	17 null spreads
4. Verify FBS membership	Keep a game only if **both** teams appear on that season's `/teams/fbs` list	**590 removed**
5. Require a final score	Drop games not completed or missing a score	1 removed
6. Consensus spread	Median of the valid spreads per game across sportsbooks (a median of four sportsbooks per game), from the home team's side	—
7. Exclude pick'em games	A spread of exactly 0 has no underdog, so an upset is undefined	10 excluded
8. Derive outcomes	`margin`, `home_win`, `fav_is_home`, `underdog_win` (the upset flag), `home_implied_margin`	—
9. Team strength	`sp_diff` = home minus away **prior-season** SP+ rating (`sp_lag=1`)	43 games lack a prior rating
Result	**3,900 games** saved to `data/clean/games.parquet`	

Why the median spread? One stale or mistyped line moves a mean but not a median. With about four sportsbooks per game, the median is the market consensus.

The integrity audit. Every build writes `results/audit.json` with a count for each check, so problems are seen instead of silently absorbed. One check is a **gate**: the share of games won by

the favorite must fall between 60% and 90%. It was 72.9%. If the spread's sign convention were ever flipped (favorites winning about 27%), the build stops before any analysis runs.

What the audit found: the API filter bug. The request `/games?classification=fbs` returned 590 rows (13.1%) whose teams were not all on the FBS list. Tabulating the raw `homeClassification` and `awayClassification` fields showed why: 589 were FBS teams hosting FCS opponents. The server-side filter keeps games with *at least one* FBS team, not games where *both* teams are FBS. The fix (step 4) checks every game against the independent `/teams/fbs` list instead of trusting the filter.

What design prevented: look-ahead leakage. `/ratings/sp?year=Y` returns ratings computed *after* season Y ends, from season Y's own games. Using them to "predict" season Y would leak the answers into the features. The pipeline uses season Y–1's ratings from the start, and a unit test pins that behavior. Section 6 measures what the leak would have done.

4. Model and evaluate (`cfb/models.py`)

The split (`split()`). - **Train:** 2019, 2021 and 2022 (2,313 games). - **Test:** 2023 and 2024 (1,587 games).

Splitting by season, not at random, means no future game informs a prediction. The function raises an error if asked to train on seasons after the test seasons.

The four hypotheses:

Test	Code	Reading the output
H1: does the spread predict winners?	`fit_spread_logit()` (statsmodels `Logit`)	Slope −0.109 per point, z = −21.3: decisive
H2: do other features add anything?	Two nested `fit_logit()` fits, likelihood-ratio test	LR = 11.2 on 5 df, p = 0.048: marginal, in-sample only
H3: are the probabilities calibrated?	`calibration_fit()` + `hosmer_lemeshow()` on the test seasons	Slope 1.09 ± 0.06, H–L p = 0.062: calibrated
H4: is the spread an unbiased margin forecast?	`fit_spread_ols()` (statsmodels `OLS`)	Slope −1.000, intercept −0.16 (p = 0.63): unbiased

Every logistic fit is checked for convergence (`fit_logit()` asserts it), so a failed optimization can never produce a p-value.

Out-of-sample scoring (`scores()`). Three forecasts are scored on the test seasons with scikit-learn: - **Naive baseline:** every game gets the training home-win rate. - **Spread-only logit.** - **Full logit:** spread plus prior SP+ difference, neutral site, conference game, week and over/under.

Metric	What it measures	Why it's used
Brier score	Mean squared error of the probability against the 0/1 outcome	The standard proper score for binary forecasts; rewards both accuracy and honest confidence
Brier skill score	1 – Brier(model) / Brier(naive)	Turns Brier into "percent better than the baseline"
Log loss	Penalizes confident misses harshly	A second proper score to confirm the ranking
AUC	How well the forecast ranks winners above losers	Discrimination, separate from calibration
Reliability diagram	Predicted against observed frequency by bin	Calibration you can see

Is the full model really no better? (`cluster_bootstrap_diff()`). The per-game Brier differences (full minus spread) are resampled 2,000 times. Whole weeks are resampled together (31 season-weeks), because games in the same week share conditions and are not independent. The resulting 95% CI of [−0.0010, +0.0028] includes zero, so there is no evidence the features help.

5. Simulate (`cfb/simulate.py`)

The question: if outcomes were nothing but the market line plus chance, would the real seasons look any different?

Two null models, both fit on training data only: 1. **Outcome redraw.** Each test game's winner is drawn with probability equal to the spread-only model's forecast. 2. **Margin redraw.** Each game's margin is −spread + Normal(0, 15.43), where 15.43 points is the training SD of the margin around the line.

Implementation. The 10,000 trials run as NumPy array operations, in chunks of 1,000 trials × 1,587 games to bound memory, with a fixed seed (42). Each trial records the upset count, the Brier score and the residual SD. `compare()` places each observed value in its simulated distribution and returns the percentile and a two-sided empirical p-value.

Reading the output: - **Upsets:** 444 observed against a simulated 95% range of 405–472 (p = 0.77). - **Residual SD:** 15.30 observed against a simulated 14.89–15.97 (p = 0.63). - **Brier:** under the line-plus-noise model, even a forecaster that knew the true probabilities would expect a Brier of about 0.182 (95% range 0.174–0.191). The market's 0.177 sits inside that range.

Nothing is unusual, so the remaining variation is consistent with irreducible randomness.

6. Verify: controlled reruns (`run_study.py`)

Two functions rerun the analysis with one thing changed, to measure what each threat to validity would have done.

`leakage_impact()`: the look-ahead leak. This rebuilds the dataset with same-season ratings (`sp_lag=0`). Both arms are scored on the same 3,857 games with the same split and seed. - With the leak, the full model appears to beat the market: Brier −0.025, CI [−0.032, −0.019]. - Its skill over the naive baseline rises from 27% to 38%. - The clean pipeline shows no edge.

The leak would have reversed the central conclusion.

`bug_impact()`: the API filter bug. This rebuilds the dataset trusting the API filter (FCS games kept). - Brier skill is inflated by 4.7 points. - The test-season upset rate falls from 28.0% to 24.9%. - The calibration test falsely rejects (p = 0.029).

A matched comparison on the shared FBS games shows that the fitted model itself is unaffected (Brier difference −0.00003). So the bug corrupted the *evaluation*, not the model.

Other verification in place: - **Tests:** 20 pytest tests cover the spread sign, the median consensus, deduplication, the FBS filter, pick'em exclusion, the leak guard, the Brier and Hosmer–Lemeshow formulas, the cluster bootstrap, and an end-to-end run on synthetic data where the true answer is known. - **Independent code reviews:** these caught and fixed, among other things, a wrong noise SD in the margin model and mismatched rows between arms of the leakage comparison. - **Byte-identical reruns:** the run is deterministic, and library versions plus a data checksum are recorded.

7. Interpret: from numbers to claims

Each claim in the report traces to a key in `results.json`. The reasoning chain:

1. **The spread carries the signal.** H1 is rejected decisively.
2. **The spread is unbiased and calibrated.** H4's slope is −1.000, and H3's slope is about 1 with H−L not rejected. The market's probabilities mean what they say.
3. **Nothing public adds to it out of sample.** H2's in-sample p = 0.048 does not survive the test seasons, and the bootstrap CI includes zero. A marginal in-sample result that fails out of sample is the signature of fitting noise.
4. **What remains is chance.** Upset counts, residual spread and Brier all sit inside their simulated distributions under a "line plus noise" world.
5. **The conclusion is robust to the two threats that could have overturned it.** Both were measured with controlled reruns.

Language discipline. "Consistent with irreducible randomness" is the claim the evidence supports; "proved random" would overstate it. A Monte Carlo can only fail to find a departure from the null; it cannot prove the null.

8. Visualize and report

Figures (`make_figures()` in `run_study.py`, matplotlib, Agg backend):

Figure	Shows	Why this form
`reliability_diagram.png`	Predicted against observed home-win rate, 10 bins, both models	The standard calibration plot; the diagonal is perfect calibration
`upset_rate_by_spread.png`	Observed against predicted upset rate in six spread buckets	Turns calibration into the audience's question: how often do 7-point underdogs win?
`mc_upsets.png`	Histograms of 10,000 simulated upset counts with the observed count marked	Shows "is this unusual?" at a glance
`mc_brier.png`	Simulated Brier distributions, including the oracle Brier from the margin model, with the market's score marked	Shows the market's score is inside the range chance alone produces
`residuals.png`	Residual histogram, and actual margin against market margin	Shows the residuals are roughly normal and unbiased

Report build (`report/build_pdf.py`). - `report.md` is converted to HTML (`markdown`, with tables and code blocks), wrapped in a print stylesheet (letter page, serif body, page numbers), and printed to PDF by headless Microsoft Edge. - The script waits for the file to finish writing, then reads the page count with `pypdf`. - This walkthrough is built the same way: `report/build_pdf.py pipeline-walkthrough.md`.

Why Markdown and not Word? The text sits in git next to the code, so every change to a number or a claim is a reviewable diff, and the PDF is rebuilt in one command.

9. Reproduce it

```
py -3.12 -m venv .venv
.venv/Scripts/python -m pip install -r requirements.txt
# create .env containing CFBD_API_KEY=<your key>
.venv/Scripts/python run_study.py --seasons 2019,2021-2024 --seed 42 --trials 10000
.venv/Scripts/python -m pytest -q
.venv/Scripts/python report/build_pdf.py
.venv/Scripts/python report/build_pdf.py pipeline-walkthrough.md
```